

অধ্যায় ৬.১ — সম্পূর্ণ নোট

Frequent Patterns, Associations এবং Correlations — বেসিক কনসেপ্ট

১. Market Basket Analysis — মূল আইডিয়া

একজন দোকান ম্যানেজার জানতে চান — কাস্টমাররা কোন কোন জিনিস একসাথে কেনেন? যেমন কেউ দুধ কিনলে সে কি সাথে রুটিও কেনে? এই প্যাটার্ন খুঁজে বের করাকেই বলে Market Basket Analysis।

ব্যবহার	ব্যাখ্যা
শেলফ প্লেসমেন্ট	যেসব জিনিস একসাথে কেনা হয় (যেমন কম্পিউটার + অ্যান্টিভাইরাস), সেগুলো কাছাকাছি রাখলে সেল বাড়ে
উল্টো স্ট্র্যাটেজি	সম্পর্কিত জিনিস ইচ্ছাকৃতভাবে দূরে রাখলে কাস্টমার হাঁটার পথে আরও জিনিস দেখে কিনতে পারে
সেল/ছাড় পরিকল্পনা	প্রিন্টার আর কম্পিউটার একসাথে কেনা হলে, প্রিন্টারে ছাড় দিলে দুটোরই সেল বাড়ে

Association Rule-এর একটা উদাহরণ:

$computer \Rightarrow antivirus\ software$ [support = 2%, confidence = 60%]

- Support (2%) — মোট ট্রানজেকশনের ২%-এ কম্পিউটার আর অ্যান্টিভাইরাস দুটোই একসাথে কেনা হয়েছে (rule-টা কতটা useful/common, তা বোঝায়)
- Confidence (60%) — যারা কম্পিউটার কিনেছে তাদের মধ্যে ৬০% অ্যান্টিভাইরাসও কিনেছে (rule-টা কতটা নিশ্চিত/certain, তা বোঝায়)

একটা rule তখনই strong ধরা হয়, যখন এটা minimum support threshold ও minimum confidence threshold — দুটোই পার করে।

২. বেসিক নোটেশন ও সংজ্ঞা

- $I = \{I_1, I_2, \dots, I_m\}$ — সব আইটেমের সেট
- D — ডাটাবেজ, যেখানে প্রতিটা transaction T হলো I -এর একটা subset
- প্রতিটা transaction-এর একটা identifier থাকে, যাকে TID বলে

Association Rule

$A \Rightarrow B$ — যেখানে A ও B দুইটা disjoint itemset (কোনো কমন আইটেম নেই)।

Support:

$$support(A \Rightarrow B) = P(A \cup B)$$

মানে A ও B দুটোই আছে এমন transaction-এর শতকরা হার।

Confidence:

$$\text{confidence}(A \Rightarrow B) = P(B|A) = \text{support_count}(A \cup B) / \text{support_count}(A)$$

মানের কাছে A আছে, তাদের মধ্যে কতজনের কাছে B-ও আছে। যেসব rule দুটো threshold-ই (min_sup, min_conf) পার করে, সেগুলোকে বলে strong rule।

৩. গুরুত্বপূর্ণ টার্মগুলো — উদাহরণ সহ

ধরুন আমাদের কাছে এই ছোট্ট দোকানের ডেটা আছে:

কাস্টমার ১: দুধ, রুটি
কাস্টমার ২: দুধ, রুটি, ডিম
কাস্টমার ৩: দুধ, রুটি
কাস্টমার ৪: ডিম

টার্ম	মানে	উদাহরণ
k-itemset	ঠিক k টা আইটেম নিয়ে গঠিত itemset	{দুধ} → 1-itemset, {দুধ, রুটি} → 2-itemset
Support count	কয়টা transaction-এ itemset-টা এসেছে	{দুধ, রুটি} এসেছে ৩ বার → count = 3
Relative support	percentage আকারে support	$3/4 = 75\%$
Frequent itemset	যার support minimum threshold পার করে	min_sup=50% হলে, {দুধ, রুটি}(75%) frequent
L_k	frequent k-itemset-দের সেট	$L_1 = \{\{\text{দুধ}\}, \{\text{রুটি}\}, \{\text{ডিম}\}\}$

সংক্ষেপে একসাথে: "{দুধ, রুটি} একটা 2-itemset (k=2), যার support count = 3, relative support = 75%। থ্রেশহোল্ড 50% এর বেশি বলে এটা frequent, আর এটা যাবে L_2 লিস্টে।"

৪. একটা বিরাট সমস্যা: Frequent Itemset-এর সংখ্যা explode করে

ধরুন একটা frequent itemset আছে ১০০টা আইটেম নিয়ে: $\{a_1, a_2, \dots, a_{100}\}$ । তাহলে এর ভেতরেই আছে $C(100,1)=100$ টা 1-itemset, $C(100,2)$ টা 2-itemset, ... এভাবে চলতেই থাকবে।

মোট সংখ্যা:

$$2^{100} - 1 \approx 1.27 \times 10^{30}$$

এটা এত বিশাল সংখ্যা যে কোনো কম্পিউটারেই গণনা বা স্টোর করা সম্ভব না! এই সমস্যা সমাধানের জন্য দুইটা নতুন কনসেপ্ট আনা হলো — Closed Itemset ও Maximal Frequent Itemset।

৫. Closed Itemset ও Maximal Itemset — গভীরভাবে বোঝা

"Superset" আগে বুঝি

Superset = অরিজিনাল সেটের সবকিছু + কিছু এক্সট্রা জিনিস। উদাহরণ: {আম} এর একটা superset হলো {আম, কাঁঠাল} — কারণ এতে আমও আছে, প্লাস এক্সট্রা কাঁঠালও আছে।

সংজ্ঞা

Closed itemset: X কে closed বলা হয়, যদি D-তে এমন কোনো super-itemset Y ($X \subset Y$) না থাকে, যার support count X-এর সমান।

Closed frequent itemset = closed + frequent, দুটোই একসাথে।

Maximal frequent itemset: X কে maximal frequent বলা হয়, যদি X frequent হয়, আর এমন কোনো super-itemset Y ($X \subset Y$) না থাকে, যেটাও frequent।

উদাহরণ দিয়ে বোঝা (ছোট ডেটাসেট)

ধরুন ডেটাবেজে ৩টা ট্রানজেকশন:

T1: আম, কাঁঠাল, লিচু
T2: আম, কাঁঠাল
T3: আম, কাঁঠাল

Itemset	Support count
{আম}	3
{কাঁঠাল}	3
{লিচু}	1
{আম, কাঁঠাল}	3
{আম, লিচু}	1
{কাঁঠাল, লিচু}	1
{আম, কাঁঠাল, লিচু}	1

min_sup = 1 ধরি (সবকিছুই frequent)।

Closed itemset বের করার প্রশ্ন: "এই itemset-কে বড় করলে count কি একই থাকে, নাকি কমে যায়?"

- count একই থাকলে → closed না

- count কমে গেলে (বা বড় করার সুযোগ না থাকলে) $\rightarrow$ closed $\checkmark$
- {আম} (count=3) $\rightarrow$ বড় করে {আম,কাঁঠাল} করলে count=3 (একই!) $\rightarrow$ closed না
- {আম,কাঁঠাল} (count=3) $\rightarrow$ বড় করে {আম,কাঁঠাল,লিচু} করলে count=1 (কমে গেছে!) $\rightarrow$ closed $\checkmark$
- {লিচু} (count=1) $\rightarrow$ বড় করে {আম,লিচু} করলে count=1 (একই!) $\rightarrow$ closed না
- {আম,কাঁঠাল,লিচু} $\rightarrow$ আর বড়ই করা যায় না $\rightarrow$ automatically closed $\checkmark$

ফলাফল: Closed itemsets = {আম,কাঁঠাল}: 3 এবং {আম,কাঁঠাল,লিচু}: 1

Maximal itemset বের করার প্রশ্ন: "এই itemset-কে বড় করলে সেটা কি এখনও frequent থাকে?"

- {আম,কাঁঠাল} $\rightarrow$ বড় করলে {আম,কাঁঠাল,লিচু}, যেটাও frequent $\rightarrow$ maximal না
- {আম,কাঁঠাল,লিচু} $\rightarrow$ আর বড় করা যায় না $\rightarrow$ maximal $\checkmark$

ফলাফল: Maximal itemset = শুধু {আম,কাঁঠাল,লিচু}: 1

৬. বইয়ের মূল উদাহরণ (Example 6.2) — বড় সংখ্যায়

ডেটাবেজে মাত্র ২টা ট্রানজেকশন:

ট্রানজেকশন ১: a1, a2, ..., a50, a51, ..., a100	(মোট ১০০টা আইটেম)
ট্রানজেকশন ২: a1, a2, ..., a50	(মোট ৫০টা আইটেম)

min_sup = 1 ধরি।

Count বের করি:

- {a1,...,a50} — উভয় ট্রানজেকশনেই আছে $\rightarrow$ count = 2
- {a1,...,a100} — শুধু ট্রানজেকশন ১-এ আছে $\rightarrow$ count = 1

Closed বের করা: {a1,...,a50}-কে বড় করে {a1,...,a100} বানালে count 2 থেকে 1-এ কমে যায়! তাই {a1,...,a50} নিজেই closed। {a1,...,a100}-কে আর বড় করা যায় না, তাই এটাও closed।

$$C = \{ \{a1,...,a100\}: 1, \{a1,...,a50\}: 2 \}$$

Maximal বের করা: {a1,...,a50}-কে বড় করলে {a1,...,a100}, যেটাও frequent (count=1) থাকে। তাই maximal না। {a1,...,a100}-কে আর বড় করা যায় না, তাই এটাই maximal।

$$M = \{ \{a1,...,a100\}: 1 \}$$

প্রমাণ — Closed কেন বেশি তথ্যবহুল

প্রশ্ন: "{a2, a45}-এর count কত?" (a2, a45 দুটোই ১-৫০ এর মধ্যে পড়ে)

- Closed set (C) দিয়ে: {a2,a45} হলো {a1,...,a50}:2-এর sub-itemset, তাই count = 2 — সঠিক উত্তর ✓
- Maximal set (M) দিয়ে: শুধু বলা যায় frequent, কিন্তু আসল count জানা যায় না X

একইভাবে, C থেকে বের করা যায়: {a8, a55}: 1 — কারণ এটা {a1,...,a100}: 1-এর sub-itemset (কিন্তু ৫০-এর সেটে নাই)।

৭. সহজ Analogy — মনে রাখার জন্য

ধরুন আপনার কাছে দুইটা রশিদ (receipt) আছে — একজন ক্রেতা ১০০ টাকার জিনিস কিনেছে, আরেকজন ৫০ টাকার (যেটার সবকিছুই প্রথম রশিদের মধ্যেও আছে)।

- Closed = দুইটা রশিদই আলাদা করে রাখা → প্রতিটার আসল টাকার পরিমাণ নির্ভুলভাবে জানা থাকে
- Maximal = শুধু বড় রশিদটা রাখা → ছোট রশিদের সঠিক পরিমাণ হারিয়ে যায়, শুধু বোঝা যায় "একটা কেনাকাটা হয়েছিল"

৮. সারমর্ম

বিষয়	মূল কথা
Association Rule Mining	দুই ধাপ: (১) frequent itemset বের করা, (২) strong rule জেনারেট করা
Support	rule-টা কতটা "common/useful"
Confidence	rule-টা কতটা "certain/নিশ্চিত"
সমস্যা	frequent itemset-এর সংখ্যা exponentially ($2^n - 1$) বেড়ে যায়
Closed itemset	কম্প্যাক্ট কিন্তু সম্পূর্ণ তথ্য সহ (exact count বের করা যায়)
Maximal itemset	সবচেয়ে কম্প্যাক্ট, কিন্তু কিছু তথ্য (exact count) হারিয়ে যায়

অধ্যায় ৬.২, ৬.২.১ ও ৬.২.২

Apriori Algorithm ও Association Rule Generation — সহজ বাংলায়

৬.২: ভূমিকা — কেন এই সেকশন?

আগের সেকশনে আমরা শিখেছি frequent itemset কী, কিন্তু কীভাবে সেটা খুঁজে বের করবো — সেটাই এই সেকশনের বিষয়। এখানে মূলত Apriori algorithm নিয়ে আলোচনা, যেটা frequent itemset খোঁজার সবচেয়ে বিখ্যাত পদ্ধতি।

৬.২.১: Apriori Algorithm

মূল আইডিয়া

Apriori একটা level-wise (ধাপে ধাপে) পদ্ধতিতে কাজ করে:

প্রথমে খুঁজি → frequent 1-itemset (L1)
তারপর সেটা দিয়ে খুঁজি → frequent 2-itemset (L2)
তারপর সেটা দিয়ে খুঁজি → frequent 3-itemset (L3)
... এভাবে চলতে থাকে যতক্ষণ না নতুন কিছু পাওয়া যায়

প্রতিটা ধাপে (প্রতিটা L_k বের করতে) পুরো ডাটাবেজ একবার করে scan করতে হয়।

Apriori Property (এই পুরো অ্যালগরিদমের ভিত্তি)

নিয়ম: যদি একটা itemset frequent হয়, তাহলে তার প্রতিটা sub-itemset (ছোট অংশ)ও frequent হবে।

উল্টো করে বললে: যদি একটা ছোট itemset frequent না হয়, তাহলে তাকে নিয়ে বানানো কোনো বড় itemset-ই frequent হতে পারবে না।

কেন এটা সত্য? ভাবুন — {রুটি, মাখন} যদি ৫ জন কিনে থাকে, তাহলে শুধু {রুটি} অন্তত ৫ জনই কিনেছে (হয়তো আরও বেশি মানুষও কিনেছে যারা মাখন কেনেনি)। মানে বড় সেটের কাউন্ট কখনো ছোট সেটের কাউন্টের চেয়ে বেশি হতে পারে না।

এই property-টা কেন কাজে লাগে? এটা দিয়ে আমরা অনেক অকেজো itemset আগেই বাদ দিয়ে দিতে পারি, পুরো ডাটাবেজ চেক না করেই — এতে অনেক সময় বাঁচে।

দুইটা ধাপ: Join আর Prune

L_{k-1} থেকে C_k (candidate k -itemset) বানাতে দুইটা কাজ করা হয়:

১) Join Step (জোড়া লাগানো)

L_{k-1} এর দুইটা itemset কে একসাথে জোড়া লাগিয়ে বড় (k -itemset) বানানো হয়, যদি তাদের প্রথম $(k-2)$ টা আইটেম একই হয়।

২) Prune Step (ছাঁটাই করা)

যে candidate itemset-এর কোনো $(k-1)$ -সাইজের sub-itemset L_{k-1} তে নেই (মানে frequent না), সেটাকে বাদ দিয়ে দেওয়া হয় — Apriori property অনুযায়ী।

পুরো উদাহরণ দিয়ে বুঝি (Table 6.1 — AllElectronics ডেটা)

ধরুন ৯টা ট্রানজেকশন আছে (মোট আইটেম: I1, I2, I3, I4, I5):

TID	আইটেম
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

$\text{min_sup} = 2$ ধরি (মানে অন্তত ২ বার এলেই frequent, relative হিসেবে $2/9 \approx 22\%$)।

ধাপ ১: $C_1 \rightarrow L_1$ (প্রতিটা আইটেম আলাদা করে গুনি)

ডাটাবেজ স্ক্যান করে প্রতিটা আইটেম কয়বার এসেছে গুনি:

Itemset	Count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

সবগুলোই $\text{min_sup}=2$ পার করেছে, তাই সবই L_1 -এ থাকবে (কেউ বাদ পড়েনি এই ধাপে)।

ধাপ ২: L_1 দিয়ে C_2 বানাই (Join)

L_1 -এর প্রতিটা আইটেমকে একে অপরের সাথে জোড়া লাগিয়ে সব সম্ভাব্য 2-itemset বানাই:

$$\{I1,I2\}, \{I1,I3\}, \{I1,I4\}, \{I1,I5\}, \{I2,I3\}, \{I2,I4\}, \{I2,I5\}, \{I3,I4\}, \{I3,I5\}, \{I4,I5\}$$

(এখানে prune করার কিছু নেই, কারণ 1-itemset সব subset তো আছেই)

এবার ডাটাবেজ স্ক্যান করে প্রতিটার count বের করি:

Itemset	Count	Frequent?
{I1,I2}	4	✓
{I1,I3}	4	✓

Itemset	Count	Frequent?
{1,1,4}	1	X (min_sup পার করেনি)
{1,1,5}	2	✓
{1,2,3}	4	✓
{1,2,4}	2	✓
{1,2,5}	2	✓
{1,3,4}	0	X
{1,3,5}	1	X
{1,4,5}	0	X

$L_2 = \{1,1,2\}, \{1,1,3\}, \{1,1,5\}, \{1,2,3\}, \{1,2,4\}, \{1,2,5\}$ — এই ৬টাই টিকলো।

ধাপ ৩: L_2 দিয়ে C_3 বানাই (এখানেই Apriori property-র আসল খেলা!)

Join: L_2 থেকে জোড়া লাগিয়ে পাওয়া যায়:

$\{1,1,2,3\}, \{1,1,2,5\}, \{1,1,3,5\}, \{1,2,3,4\}, \{1,2,3,5\}, \{1,2,4,5\}$

Prune (এখানেই কাজে লাগে Apriori property): প্রতিটার সব 2-item subset চেক করি — সেগুলো L_2 তে আছে কি না?

- $\{1,1,2,3\}$ এর subset: $\{1,1,2\}$ ✓, $\{1,1,3\}$ ✓, $\{1,2,3\}$ ✓ → সব L_2 -তে আছে → রাখি
- $\{1,1,2,5\}$ এর subset: $\{1,1,2\}$ ✓, $\{1,1,5\}$ ✓, $\{1,2,5\}$ ✓ → সব আছে → রাখি
- $\{1,1,3,5\}$ এর subset: $\{1,3,5\}$ X (এটা L_2 -তে নেই!) → বাদ দিই
- $\{1,2,3,4\}$ এর subset: $\{1,3,4\}$ X (নেই!) → বাদ দিই
- $\{1,2,3,5\}$ এর subset: $\{1,3,5\}$ X (নেই!) → বাদ দিই
- $\{1,2,4,5\}$ এর subset: $\{1,4,5\}$ X (নেই!) → বাদ দিই

লক্ষ্য করুন — এই ৪টা বাদ দেওয়া হলো ডাটাবেজ স্ক্যান না করেই, শুধু আগের ধাপের ফলাফল দেখে! এটাই Apriori-র আসল শক্তি — একেজো candidate আগেই ফেলে দেওয়া, সময় বাঁচানো।

তাহলে $C_3 = \{1,1,2,3\}, \{1,1,2,5\}$ — মাত্র ২টা টিকলো।

এবার ডাটাবেজ স্ক্যান করে দেখি:

Itemset	Count
{1,1,2,3}	2 ✓
{1,1,2,5}	2 ✓

দুটোই min_sup পার করেছে, তাই $L_3 = \{I1, I2, I3\}, \{I1, I2, I5\}$ ।

ধাপ ৪: L_3 দিয়ে C_4 বানানোর চেষ্টা

Join করলে পাওয়া যায় শুধু: $\{I1, I2, I3, I5\}$

কিন্তু Prune করার সময় দেখা যায়, এর একটা subset $\{I2, I3, I5\}$ frequent না (আমরা আগেই এটা বাদ দিয়েছিলাম C_3 থেকে)।

তাই $\{I1, I2, I3, I5\}$ -ও বাদ। $C_4 =$ খালি (ϕ)।

যেহেতু C_4 খালি, অ্যালগরিদম এখানেই থেমে যায়। সব frequent itemset পাওয়া হয়ে গেছে:

$$L = L1 \cup L2 \cup L3$$

সংক্ষেপে Apriori Algorithm-এর Pseudocode

১. L_1 বের করো (প্রতিটা আইটেম আলাদাভাবে গুনে)
২. $k = 2$ থেকে শুরু করে, যতক্ষণ L_{k-1} খালি না হয়:
 - ক) apriori_gen() দিয়ে L_{k-1} থেকে candidate C_k বানাও (Join + Prune)
 - খ) ডাটাবেজ স্ক্যান করে C_k এর প্রতিটার count বের করো
 - গ) যেগুলো min_sup পার করে, সেগুলো নিয়ে L_k বানাও
৩. সব $L_1, L_2, L_3 \dots$ মিলিয়ে ফাইনাল আনসার L দাও

apriori_gen প্রসিডিউর-টাই মূলত join আর prune-এর কাজ করে, আর has_infrequent_subset চেক করে কোনো candidate-এর subset frequent কি না।

৬.২.২: Frequent Itemsets থেকে Association Rule জেনারেট করা

মূল আইডিয়া

আমরা আগেই সব frequent itemset পেয়ে গেছি। এখন সেখান থেকে strong association rule বানাতে হবে।

নিয়ম:

- প্রতিটা frequent itemset I নাও
- I -এর সব nonempty subset s বানাও
- প্রতিটা s -এর জন্য rule বানাও: $s \Rightarrow (I - s)$
- এই rule-এর confidence চেক করো (নিচের সূত্র অনুযায়ী)

$$confidence = support_count(I) / support_count(s)$$

যদি এটা min_conf পার করে, তাহলে rule-টা রাখো (এটাই strong rule)।

গুরুত্বপূর্ণ পয়েন্ট: যেহেতু rule-গুলো frequent itemset থেকেই বানানো হচ্ছে, তাই support অটোম্যাটিক্যালি satisfy হয়ে যায়। শুধু confidence চেক করলেই হবে।

উদাহরণ দিয়ে বুঝি (একই ডেটা থেকে)

ধরুন আমরা frequent itemset $X = \{I1, I2, I5\}$ নিয়ে rule বানাতে চাই।

ধাপ ১: X-এর সব nonempty subset বের করি:

$$\{I1, I2\}, \{I1, I5\}, \{I2, I5\}, \{I1\}, \{I2\}, \{I5\}$$

ধাপ ২: প্রতিটা subset s দিয়ে rule বানাই $s \Rightarrow (X - s)$, আর confidence বের করি। মনে আছে, আগের হিসাব থেকে:

- $\{I1, I2, I5\}$ এর count = 2
- $\{I1, I2\}$ এর count = 4, $\{I1, I5\}$ এর count = 2, $\{I2, I5\}$ এর count = 2
- $\{I1\}$ এর count = 6, $\{I2\}$ এর count = 7, $\{I5\}$ এর count = 2

Rule	হিসাব	Confidence
$\{I1, I2\} \Rightarrow I5$	2/4	50%
$\{I1, I5\} \Rightarrow I2$	2/2	100%
$\{I2, I5\} \Rightarrow I1$	2/2	100%
$I1 \Rightarrow \{I2, I5\}$	2/6	33%
$I2 \Rightarrow \{I1, I5\}$	2/7	29%
$I5 \Rightarrow \{I1, I2\}$	2/2	100%

ধাপ ৩: ধরি $\text{min_conf} = 70\%$ । তাহলে শুধু যেগুলো ৭০%-এর বেশি, সেগুলোই strong rule হিসেবে রাখা হবে:

- ☒ $\{I1, I5\} \Rightarrow I2$ (confidence 100%)
- ☒ $\{I2, I5\} \Rightarrow I1$ (confidence 100%)
- ☒ $I5 \Rightarrow \{I1, I2\}$ (confidence 100%)

বাকি তিনটা (50%, 33%, 29%) বাদ পড়ে গেলো, কারণ থ্রেশহোল্ড পার করেনি।

একটা মজার পয়েন্ট

সাধারণ classification rule-এ ডান দিকে (right side) একটাই জিনিস থাকে, কিন্তু association rule-এ ডান দিকে একাধিক আইটেমও থাকতে পারে — যেমন উপরের উদাহরণে " $I5 \Rightarrow \{I1, I2\}$ "।

সংক্ষেপে সব একসাথে

বিষয়	মূল কথা
৬.২ (ভূমিকা)	কীভাবে frequent itemset বের করা যায়, তার পদ্ধতি নিয়ে আলোচনা
৬.২.১ (Apriori)	Level-wise পদ্ধতি: $L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow \dots$; Join+Prune দিয়ে candidate কমানো হয় Apriori property ব্যবহার করে
৬.২.২ (Rule জেনারেশন)	প্রতিটা frequent itemset-এর subset নিয়ে rule বানানো হয়, confidence চেক করে strong rule বাছাই করা হয়

মূল শিক্ষা: Apriori-র আসল শক্তি হলো — "একটা ছোট জিনিস frequent না হলে, তার বড় ভাঙ্গনও frequent হতে পারবে না" — এই সহজ যুক্তি দিয়ে অনেক অকেজো candidate আগেই বাদ দিয়ে সময় বাঁচানো যায়, বারবার পুরো ডাটাবেজ স্ক্যান না করেই।

অধ্যায় ৬.২.৩ – ৬.৩.২

Frequent Pattern Mining: Efficiency, FP-growth, Vertical Format ও Pattern Evaluation

সহজ বাংলায়, উদাহরণসহ সম্পূর্ণ নোট

৬.২.৩: Apriori-র Efficiency বাড়ানোর পদ্ধতিগুলো

সমস্যাটা কী ছিল?

Apriori algorithm ভালো কাজ করে, কিন্তু এর দুইটা বড় সমস্যা আছে:

- অনেক candidate itemset বানাতে হয় (এখনও অনেক)
- বারবার পুরো ডাটাবেজ scan করতে হয় — প্রতিটা L_k বের করতে একবার করে scan লাগে

এই সমস্যা কমানোর জন্য কয়েকটা কৌশল বের হয়েছে। একটা একটা করে বুঝি।

১) Hash-based Technique (হ্যাশ-ভিত্তিক পদ্ধতি)

মূল আইডিয়া: L_1 বের করার সময়ই (প্রথম scan-এই), সাথে সাথে সব সম্ভাব্য 2-itemset গুলোকে একটা hash table-এর বিভিন্ন "বাক্সে" (bucket) ফেলে দেওয়া হয়, আর প্রতিটা বাক্সের count বাড়ানো হয়।

কীভাবে কাজে লাগে? যে বাক্সে count খুবই কম ($\min_sup$ -এর নিচে), সেই বাক্সে যেসব 2-itemset পড়েছিল, সেগুলো অবশ্যই frequent হতে পারবে না — তাই তাদের C_2 থেকে বাদ দেওয়া যায়, ডাটাবেজ আবার scan না করেই।

সহজ analogy: ধরুন আপনি ৫০ জনের একটা পরীক্ষার খাতা ৫টা বাক্সে ভাগ করে রাখলেন (রোল নম্বর অনুযায়ী)। যদি কোনো একটা বাক্সে মাত্র ১টা খাতা থাকে, আপনি বুঝে যাবেন সেই বাক্সে "পাস করা" খাতা কম থাকার সম্ভাবনা বেশি — বাকি বাক্সগুলো আগে চেক করবেন।

২) Transaction Reduction (ট্রানজেকশন কমানো)

মূল আইডিয়া: যদি একটা transaction-এ কোনো frequent k-itemset না থাকে, তাহলে সেই transaction-এ কোনো frequent (k+1)-itemset-ও থাকতে পারবে না (Apriori property অনুযায়ী)।

কী করা হয়? এমন transaction গুলোকে ভবিষ্যতের স্ক্যানগুলো থেকে বাদ দিয়ে/মার্ক করে দেওয়া হয় — পরের বার আর সেগুলো চেক করতে হয় না।

সহজ ভাষায়: ধরুন কোনো কাস্টমারের বাক্সেটে এমন কিছুই নেই যেটা এখন পর্যন্ত frequent হিসেবে চিহ্নিত হয়েছে। তার মানে সেই বাক্সেট নিয়ে আর ভবিষ্যতে মাথা ঘামানোর দরকার নেই — বাদ দিয়ে দিন।

৩) Partitioning (ডাটা ভাগ করা)

এই পদ্ধতিতে পুরো ডাটাবেজ মাত্র ২ বার scan করলেই কাজ চলে যায়। দুইটা ধাপ:

Phase I:

- ডাটাবেজকে n টা ছোট ছোট ভাগে (partition) ভাগ করা হয়
- প্রতিটা ভাগে আলাদাভাবে frequent itemset (local frequent itemset) বের করা হয় — এতে ১ বার scan লাগে

একটা গুরুত্বপূর্ণ প্রপারটি: পুরো ডাটাবেজে যদি কোনো itemset potentially frequent হয়, তাহলে সেটা অন্তত একটা partition-এ local frequent হবেই। তাই সব partition-এর local frequent itemset গুলো মিলিয়ে একটা "global candidate list" বানানো হয়।

Phase II:

- এবার পুরো ডাটাবেজ আরেকবার scan করে দেখা হয় — সেই candidate গুলো আসলেই পুরো ডাটাবেজে frequent কি না

সহজ analogy: ধরুন একটা বড় শহরের ৫টা এলাকায় জরিপ করছেন কোন খাবার সবচেয়ে জনপ্রিয়। প্রতিটা এলাকায় আলাদা করে টপ খাবারগুলো বের করলেন (প্রথম রাউন্ড)। এরপর সব এলাকার টপ খাবারগুলো একসাথে করে, পুরো শহর জুড়ে সেগুলোর আসল জনপ্রিয়তা যাচাই করলেন (দ্বিতীয় রাউন্ড)। পুরো ডাটাবেজ বারবার scan করার দরকার নেই।

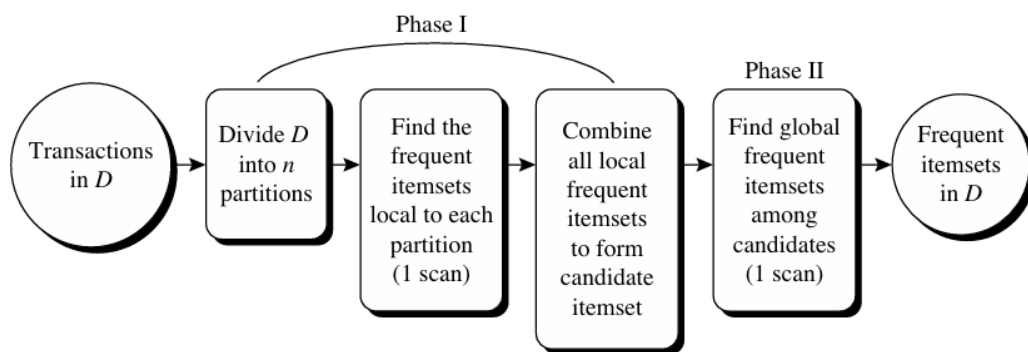


Figure 6.6 Mining by partitioning the data.

৪) Sampling (নমুনা নিয়ে কাজ করা)

মূল আইডিয়া: পুরো ডাটাবেজ D না নিয়ে, তার একটা ছোট random sample S নেওয়া হয়। এই sample-টা এত ছোট রাখা হয় যাতে পুরোটা মেমোরিতে বসে যায় (তাহলে ডিস্ক থেকে বারবার পড়তে হয় না)।

সমস্যা: যেহেতু পুরো ডাটাবেজ না দেখে শুধু sample দেখা হচ্ছে, তাই কিছু global frequent itemset হয়তো miss হয়ে যেতে পারে।

সমাধান: sample-এ খোঁজার সময় আসল min_sup -এর চেয়ে একটু কম threshold ব্যবহার করা হয় (যাতে বেশি জিনিস ধরা পড়ে, কম মিস হয়)। এরপর বাকি ডাটাবেজ দিয়ে সেগুলোর আসল count যাচাই করা হয়।

সহজ ভাষায়: পুরো দেশের মানুষের মতামত না নিয়ে, একটা ছোট sample survey করা, কিন্তু একটু সতর্কভাবে (threshold কম রেখে) যাতে গুরুত্বপূর্ণ কিছু বাদ না পড়ে।

৫) Dynamic Itemset Counting (গতিশীল গণনা)

মূল আইডিয়া: সাধারণ Apriori-তে নতুন candidate যোগ করা হয় শুধু প্রতিটা সম্পূর্ণ scan শেষ হওয়ার পরই। কিন্তু এই পদ্ধতিতে ডাটাবেজকে কয়েকটা ব্লকে ভাগ করে, যেকোনো স্টার্ট পয়েন্টে নতুন candidate যোগ করা যায় — পুরো scan শেষ হওয়ার অপেক্ষা করতে হয় না।

ফলাফল: Apriori-র চেয়ে কম সংখ্যক ডাটাবেজ scan লাগে।

৬.২.৩ এর সারসংক্ষেপ

পদ্ধতি	মূল কৌশল
Hash-based	Hash bucket দিয়ে একেজো 2-itemset আগেই বাদ
Transaction reduction	একেজো transaction ভবিষ্যতের scan থেকে বাদ
Partitioning	ছোট ভাগে ভাগ করে, মাত্র ২ বার scan-এই কাজ শেষ
Sampling	পুরো ডাটার বদলে ছোট নমুনা নিয়ে কাজ (কম threshold দিয়ে)
Dynamic itemset counting	scan এর মাঝেই নতুন candidate যোগ করা

৬.২.৪: FP-Growth — Candidate Generation ছাড়াই Mining

Apriori-র আসল সমস্যা কী?

Apriori-র দুইটা বড় খরচ (cost):

- প্রচুর candidate সেট বানাতে হয় — যদি 10^4 টা frequent 1-itemset থাকে, তাহলে Apriori-কে 10^7 -এর বেশি candidate 2-itemset বানাতে হবে!
- বারবার পুরো ডাটাবেজ scan করে প্রতিটা candidate-এর count মিলাতে হয় — এটা অনেক সময় নেয়

প্রশ্ন: এমন কোনো পদ্ধতি আছে কি, যেখানে candidate বানানোর এই ভারী প্রক্রিয়াটাই লাগবে না?

উত্তর: হ্যাঁ — FP-growth (Frequent Pattern growth)

FP-growth-এর মূল কৌশল (Divide and Conquer)

- প্রথমে পুরো ডাটাবেজকে একটা কম্প্যাক্ট গাছে (FP-tree) সংকুচিত করা হয়
- এই গাছটাকে ছোট ছোট conditional database-এ ভাগ করা হয় — প্রতিটা একটা নির্দিষ্ট আইটেমের সাথে সম্পর্কিত
- প্রতিটা ছোট ডাটাবেজকে আলাদাভাবে mine করা হয়

এভাবে বড় সমস্যাটাকে ছোট ছোট সমস্যায় ভেঙে ফেলা হয় — আর কোনো candidate generate করার দরকারই পড়ে না!

ধাপে ধাপে উদাহরণ (একই AllElectronics ডেটা দিয়ে)

মনে করি আগের সেই ৯টা transaction, $\text{min_sup} = 2$ । প্রথমে ডেটা এবং সাজানোর ক্রম মনে করি।

আমাদের ডেটা

TID	আইটেম
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

সাজানো ক্রম: I2 > I1 > I3 > I4 > I5 (count অনুযায়ী: 7, 6, 6, 2, 2)

ধাপ ০: প্রতিটা transaction আগে সাজিয়ে নিই

TID	আসল	সাজানোর পর
T100	I1,I2,I5	I2, I1, I5
T200	I2,I4	I2, I4
T300	I2,I3	I2, I3
T400	I1,I2,I4	I2, I1, I4
T500	I1,I3	I1, I3
T600	I2,I3	I2, I3
T700	I1,I3	I1, I3
T800	I1,I2,I3,I5	I2, I1, I3, I5
T900	I1,I2,I3	I2, I1, I3

ধাপ ১-৯: গাছে একে একে transaction যোগ করা

ধাপ ১ (T100: I2,I1,I5): গাছ খালি ছিল, তাই সরলরেখার মতো প্রথম শাখা তৈরি হয়:

```

null
└─ I2:1
    └─ I1:1
        └─ I5:1
  
```

ধাপ ২ (T200: I2,I4): I2 আগে থেকেই আছে (common prefix) — নতুন শাখা না বানিয়ে count বাড়াই: I2:1 → I2:2 | I4 নতুন, তাই নতুন শাখা:

```

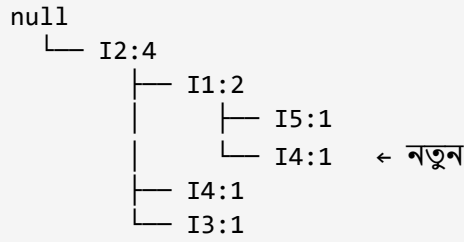
null
└─ I2:2
    └─ I1:1 → I5:1
        └─ I4:1
  
```

ধাপ ৩ (T300: I2,I3): I2 → I2:3 | I3 নতুন, তাই নতুন শাখা:

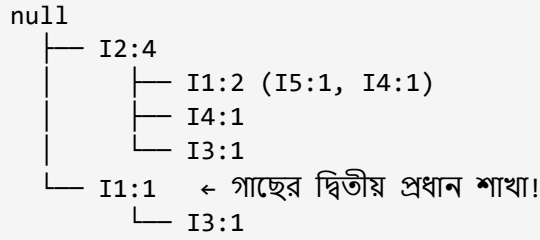
```

null
└─ I2:3
    └─ I1:1 → I5:1
        └─ I4:1
            └─ I3:1
  
```

ধাপ ৪ (T400: I2,I1,I4): I2 → I2:4 | I1 → I1:2 | I4 নতুন (I1-এর নিচে এতদিন শুধু I5 ছিল):



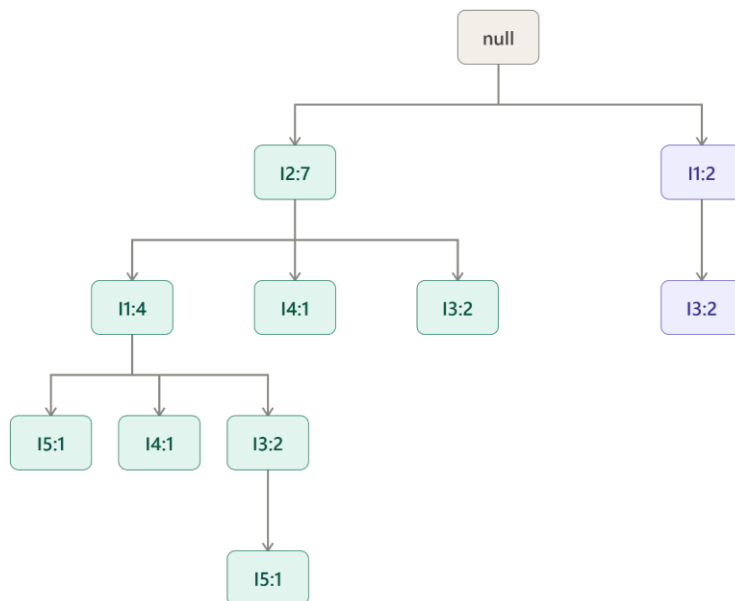
ধাপ ৫ (T500: I1,I3): এবার শুরুই হচ্ছে I1 দিয়ে — root-এর নিচে সরাসরি child হিসেবে এখন পর্যন্ত শুধু I2 আছে, তাই কোনো prefix মিলছে না। root-এর নিচে সম্পূর্ণ নতুন, আলাদা একটা শাখা তৈরি হয়:



এটাই মূল কনফিউশনের জায়গা — গাছের root-এর নিচে একের বেশি প্রধান শাখা থাকতে পারে, যদি transaction ভিন্ন আইটেম দিয়ে শুরু হয়।

- **ধাপ ৬ (T600: I2,I3):** I2 মিলে যায় → I2:5। I3-ও মিলে যায় → I3:2
- **ধাপ ৭ (T700: I1,I3):** দ্বিতীয় শাখায় I1 মিলে যায় → I1:2, I3-ও মিলে যায় → I3:2
- **ধাপ ৮ (T800: I2,I1,I3,I5):** I2→I2:6, I1→I1:3, I3 নতুন→I3:1, I5 নতুন→I5:1
- **ধাপ ৯ (T900: I2,I1,I3):** I2→I2:7, I1→I1:4, I3 মিলে যায়→I3:2

চূড়ান্ত গাছ



টিল = I2-শাখা, বেগুনি = I1-শাখা (root-এর সরাসরি সন্তান)

লক্ষ্য করুন — root-এর নিচে দুইটা প্রধান শাখা (টিল রঙের I2-শাখা, বেগুনি রঙের I1-শাখা)। ডেটাতে ৭টা transaction I2 দিয়ে শুরু হয়েছিল, আর ২টা (T500, T700) শুরু হয়েছিল I1 দিয়ে (কারণ তাদের মধ্যে I2 ছিলই না) — তাই দুইটা আলাদা প্রধান শাখা তৈরি হলো।

Node-link (চেইন) জিনিসটা কী?

I3 আইটেমটা গাছে তিন জায়গায় আছে। প্রতিবার গাছ ঘুরে খুঁজে বের করার বদলে, একটা header table রাখা হয়, যেখানে প্রতিটা আইটেমের পাশে একটা "চেইন" (node-link) থাকে, যেটা সরাসরি সেই আইটেমের সব occurrence-এ দ্রুত পৌঁছে দেয়।

মাইনিং অংশ — I5 দিয়ে শুরু (সবচেয়ে কম frequent আইটেম)

লক্ষ্য: এই গাছ থেকে I5 জড়িত সব frequent pattern বের করা।

ধাপ ১ — **occurrence খুঁজি:** I5 আছে দুই জায়গায় —

- Path 1: null → I2 → I1 → I5 (count 1)
- Path 2: null → I2 → I1 → I3 → I5 (count 1)

ধাপ ২ — **Conditional Pattern Base:** মানে হলো "I5-এর আগে কী কী ছিল" (I5 বাদ দিয়ে)।

- Path 1 থেকে I5 বাদ দিলে: {I2, I1}, weight 1
- Path 2 থেকে I5 বাদ দিলে: {I2, I1, I3}, weight 1

$$\text{Conditional pattern base} = \{I2, I1:1\}, \{I2, I1, I3:1\}$$

Analogy: ধরুন আপনি জানতে চান "যারা লিচু কিনেছে, তারা লিচু কেনার আগে আর কী কী কিনেছিল?" — শুধু সেই কাস্টমারদের বাস্কেট থেকে "লিচু" বাদ দিয়ে বাকি জিনিস লিখে রাখা — এটাই conditional pattern base।

ধাপ ৩ — ছোট conditional tree বানাই:

- I2 এর মোট weight: $1+1 = 2$ (দুই path-এই আছে)
- I1 এর মোট weight: $1+1 = 2$ (দুই path-এই আছে)
- I3 এর মোট weight: 1 (শুধু দ্বিতীয় path-এ) → $\text{min_sup}=2$ পার করেনি, বাদ

$$\text{Conditional FP-tree} = \text{একটাই সরলরেখা: } I2:2 \rightarrow I1:2$$

ধাপ ৪ — **Pattern জেনারেট করি:** যেহেতু এটা একটা মাত্র সরলরেখা, তাই সব sub-combination বানিয়ে, তার সাথে I5 আবার জুড়ে দিই:

- I2 একা → I5 জুড়ে: {I2, I5}: 2
- I1 একা → I5 জুড়ে: {I1, I5}: 2
- I2, I1 একসাথে → I5 জুড়ে: {I2, I1, I5}: 2

বাকি আইটেমগুলো (I4, I3, I1) — একই পদ্ধতি

I4-এর হিসাব (সবচেয়ে সহজ কেস)

- Path A: null $\rightarrow$ I2 $\rightarrow$ I1 $\rightarrow$ I4:1 (I4 বাদ দিলে: {I2,I1}, weight 1)
- Path B: null $\rightarrow$ I2 $\rightarrow$ I4:1 (I4 বাদ দিলে: {I2}, weight 1)

$$\text{Conditional pattern base} = \{I2, I1:1\}, \{I2:1\}$$

- I2 এর মোট weight = 2
- I1 এর মোট weight = 1 $\rightarrow$ min_sup=2 পার করেনি, বাদ

$$\text{Conditional tree} = \langle I2:2 \rangle \rightarrow \text{ফলাফল: } \{I2, I4\}: 2$$

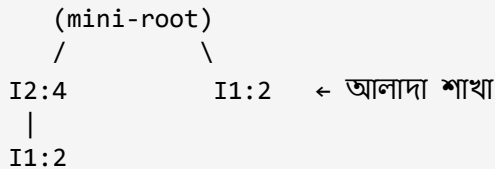
লক্ষ্য করুন — I1 বাদ পড়ে যাওয়ায় {I1,I4} বা {I2,I1,I4} কোনোটাই পাওয়া গেলো না।

I3-এর হিসাব (একটু জটিল — এখানেই আসল মজা)

- Path 1: null $\rightarrow$ I2 $\rightarrow$ I3:2 (I3 বাদ দিলে: {I2}, weight 2)
- Path 2: null $\rightarrow$ I2 $\rightarrow$ I1 $\rightarrow$ I3:2 (I3 বাদ দিলে: {I2,I1}, weight 2)
- Path 3: null $\rightarrow$ I1 $\rightarrow$ I3:2 (I3 বাদ দিলে: {I1}, weight 2)

$$\text{Conditional pattern base} = \{I2, I1:2\}, \{I2:2\}, \{I1:2\}$$

এবার এই তিনটা prefix দিয়ে ছোট গাছ বানাই (এখানেই single path হয় না!) — {I2:2} আর {I2,I1:2} দুটোই I2 দিয়ে শুরু, তাই merge হয়ে যায় (I2 count = 4), আর {I1:2} আলাদা শাখা:



এটাই {I2:4, I1:2}, {I1:2} — মানে দুইটা আলাদা branch (single path না)।

এবার প্রতিটা আইটেমের মোট count আলাদাভাবে বের করে I3-এর সাথে জোড়া লাগাতে হয়:

- I2-এর মোট count = 4 $\rightarrow$ {I2, I3}: 4
- I1-এর মোট count = 2+2 = 4 $\rightarrow$ {I1, I3}: 4
- I2 আর I1 একসাথে (একই পথে) = 2 $\rightarrow$ {I2, I1, I3}: 2

$$\text{ফলাফল} = \{I2, I3:4\}, \{I1, I3:4\}, \{I2, I1, I3:2\}$$

I1-এর হিসাব (সবচেয়ে সহজ কেস)

- I2-এর নিচে I1:4 (prefix: {I2}, weight 4)
- root-এর সরাসরি নিচে I1:2 — এর আগে কিছুই নেই (empty prefix), তাই গণনায় ধরা হয় না

$$\text{Conditional pattern base} = \{I2:4\} \rightarrow \text{Conditional tree} = \langle I2:4 \rangle \rightarrow \text{ফলাফল: } \{I2, I1\}: 4$$

সবগুলো একসাথে দেখলে প্যাটার্নটা ধরা পড়বে

আইটেম	গাছে পথ কতটা জটিল	Conditional tree	কেন
I5	সরল, দুই path একই দিকে	Single path	সহজে সব combination
I4	সরল, একটা আইটেম বাদ পড়ে	Single path (ছোট)	I1 threshold পার করেনি
I3	জটিল — branch ভাগ হয়ে যায়	Multi-path	প্রতিটা আইটেম আলাদাভাবে গুনতে হয়
I1	সবচেয়ে সরল, একটাই prefix	Single node	root-এর সরাসরি সন্তান বাদ পড়ে

মূল শিক্ষা: যখন conditional tree একটা মাত্র সরলরেখা (single path) হয়, তখন pattern বের করা সহজ — সব sub-combination বানিয়ে ফেলাই যথেষ্ট। কিন্তু যখন গাছ একাধিক শাখায় ভাগ হয়ে যায় (I3-এর মতো), তখন প্রতিটা আইটেমের সামগ্রিক (সব শাখা মিলিয়ে) count আলাদাভাবে বের করতে হয়।

৬.২.৫: Vertical Data Format দিয়ে Mining (Eclat)

Horizontal বনাম Vertical ফরম্যাট

এতদিন যেভাবে ডেটা দেখেছি (Apriori, FP-growth-এ), সেটা Horizontal format — প্রতিটা TID-এর সাথে তার ভেতরের আইটেমের লিস্ট:

```
T100: I1, I2, I5  
T200: I2, I4  
T300: I2, I3  
...
```

কিন্তু Vertical format-এ ব্যাপারটা উল্টো — প্রতিটা আইটেমের সাথে, সেই আইটেম কোন কোন TID-তে আছে তার লিস্ট রাখা হয়:

```
I1: T100, T400, T500, T700, T800, T900  
I2: T100, T200, T300, T400, T600, T800, T900
```

সহজ analogy: একটা ক্লাসের অ্যাটেনডেন্স খাতা। Horizontal হলো "প্রতিদিনের তারিখ ধরে কারা এসেছিল"। Vertical হলো "প্রতিটা ছাত্রের নাম ধরে সে কোন কোন দিন এসেছিল" — দুটোই একই তথ্য, শুধু গোছানোর দিক আলাদা।

পুরো টেবিলটা vertical format-এ

itemset	TID set
I1	{T100, T400, T500, T700, T800, T900}
I2	{T100, T200, T300, T400, T600, T800, T900}
I3	{T300, T500, T600, T700, T800, T900}
I4	{T200, T400}
I5	{T100, T800}

I1-এর TID set-এ 6টা TID আছে, মানে support count = 6 — শুধু TID set-এর সাইজ দেখলেই count পাওয়া যায়, আলাদা করে গণনার দরকার নেই।

2-itemset কীভাবে বের করি?

মূল কৌশল: দুইটা আইটেমের TID set-কে intersect (কমন অংশ বের) করলেই তাদের একসাথে থাকা transaction গুলো পাওয়া যাবে।

$$\{I1, I2\}\text{-এর TID set} = I1\text{-এর TID set} \cap I2\text{-এর TID set}$$

হাতে-কলমে দেখি:

$$I1 = \{T100, T400, T500, T700, T800, T900\} \quad I2 = \{T100, T200, T300, T400, T600, T800, T900\}$$

দুটোতে কমন কোনগুলো? $\rightarrow T100 \checkmark, T400 \checkmark, T800 \checkmark, T900 \checkmark$

$$\{I1, I2\} = \{T100, T400, T800, T900\} \rightarrow \text{count} = 4$$

সব 2-itemset বের করি

itemset	TID set	Count
{I1, I2}	{T100, T400, T800, T900}	4
{I1, I3}	{T500, T700, T800, T900}	4
{I1, I4}	{T400}	1
{I1, I5}	{T100, T800}	2
{I2, I3}	{T300, T600, T800, T900}	4
{I2, I4}	{T200, T400}	2
{I2, I5}	{T100, T800}	2
{I3, I5}	{T800}	1

{I1, I4} আর {I3, I5}-এর count মাত্র 1, min_sup=2 পার করছে না — তাই এই দুটো বাদ পড়ে যায়।

3-itemset — একই পদ্ধতি, Apriori property মেনে

শুধু সেই 3-itemset-দের candidate ধরা হবে, যাদের সব 2-item subset frequent। candidate তৈরি হয় শুধু দুটো: {I1, I2, I3} আর {I1, I2, I5}।

$$\{I1, I2, I3\}\text{-এর TID set} = \{I1, I2\}\text{-এর TID set} \cap \{I1, I3\}\text{-এর TID set}$$

$$\{I1, I2\} = \{T100, T400, T800, T900\}, \{I1, I3\} = \{T500, T700, T800, T900\} \rightarrow \text{কমন: } \{T800, T900\} \rightarrow \text{count} = 2$$

$$\text{একইভাবে } \{I1, I2, I5\} = \{I1, I2\} \cap \{I1, I5\} = \{T100, T800\} \rightarrow \text{count} = 2$$

সবচেয়ে বড় সুবিধা: কোনো ধাপেই আবার ডাটাবেজ scan করতে হয়নি! শুধু আগের TID set গুলো intersect করেই count পাওয়া যাচ্ছে।

Diffset — মেমোরি বাঁচানোর কৌশল

TID set গুলো যদি অনেক বড় হয়, তাহলে বারবার এত বড় সেট intersect করা সময়সাপেক্ষ ও মেমোরি-খরচে। Diffset কৌশলে পুরো TID set না রেখে শুধু পার্থক্যটুকু রাখা হয়।

$$I1 = \{T100, T400, T500, T700, T800, T900\}, \{I1, I2\} = \{T100, T400, T800, T900\}$$

I1-এ আছে কিন্তু {I1,I2}-এ নেই — এমন TID কোনগুলো? $\rightarrow \{T500, T700\}$

$$diffset(\{I1,I2\}, \{I1\}) = \{T500, T700\}$$

মানে ৪টা TID পুরোটা মনে না রেখে, শুধু ২টা "পার্থক্য" মনে রাখলেই যথেষ্ট — বিশেষ করে যখন ডেটা "dense" আর pattern লম্বা হয়, এতে মেমোরি অনেক বাঁচে।

৬.২.৬: Closed আর Max Pattern সরাসরি Mine করা

সমস্যাটা মনে করি

সব frequent itemset বের করাটা explode করে যেতে পারে ($2^{100}-1$ পর্যন্ত)। তাই আমরা চাই সরাসরি closed itemset বের করতে, পুরো frequent itemset list না বানিয়েই।

নাইভ (বাজে) পদ্ধতি: আগে সব frequent itemset বের করে, তারপর যেগুলো কোনো বড় itemset-এর subset (একই support), সেগুলো বাদ দেওয়া — কিন্তু এতে প্রথমেই $2^{100}-1$ টা itemset বানাতে হবে, যা অসম্ভব খরচ!

সমাধান: mining করার সময়ই সরাসরি বুঝে ফেলা কোনটা closed, আর একেজো শাখা mining শুরুর আগেই কেটে ফেলা (pruning)।

পদ্ধতি ১: Item Merging

নিয়ম: যদি itemset X ধারণকারী প্রতিটা transaction-এ আরেকটা itemset Y-ও থাকে (Y-এর বড় সংস্করণ ছাড়াই), তাহলে XUY সরাসরি একটা closed itemset — আলাদা করে খুঁজতে হয় না।

উদাহরণ (FP-tree থেকেই): I5-এর conditional pattern base ছিল {I2,I1}, {I2,I1,I3}। দুই path-এর প্রতিটাতেই {I2,I1} আছে (কোনো বড় সুপারসেট ছাড়া), তাই সরাসরি merge:

$$\{I5, I2, I1\}: 2 \text{ (closed itemset, সরাসরি!)}$$

এতে সুবিধা — I5-কে ঘিরে I2, I1 আছে কিনা আলাদা করে খোঁজার দরকার নেই, একবারেই merge হয়ে যায়।

পদ্ধতি ২: Sub-itemset Pruning

নিয়ম: যদি frequent itemset X আগের কোনো closed itemset Y-এর proper subset হয় আর support একই হয়, তাহলে X ও তার descendant সবগুলো বাদ দেওয়া যায়।

উদাহরণ: ডেটাবেজ {a₁,...,a₁₀₀}, {a₁,...,a₅₀}, min_sup=1। a₁ দিয়ে mining শুরু করে closed itemset পেয়ে গেছি: {a₁,...,a₅₀}: 2।

চেক করি: support({a₂}) = 2, আর support({a₁,...,a₅₀}) = 2 — একই! আর {a₂} তো {a₁,...,a₅₀}-এর proper subset।

সিদ্ধান্ত: a₂-কে নিয়ে আলাদাভাবে mining করার দরকার নেই — একই যুক্তিতে a₃,...,a₅₀ সবগুলোই বাদ। পুরো mining শেষ হয়ে যায় শুধু a₁-এর projected database mine করেই!

পদ্ধতি ৩: Item Skipping

নিয়ম: Depth-first mining-এর সময়, লোকাল frequent আইটেম p-এর support নিচের স্তরে যদি উপরের স্তরের মতোই থাকে, তাহলে p-কে উপরের স্তরের header table থেকে বাদ দেওয়া যায়।

উদাহরণ: a_2 -এর support global header table-এও 2, a_1 -এর projected database-এও 2 — একই! তাই a_2 -কে global স্তরে আর আলাদা রাখার দরকার নেই — একই যুক্তিতে $a_3, \dots, a_{50}$ -ও।

এই তিনটা কৌশলের মূল উদ্দেশ্য একই — mining শুরুর আগেই বুঝে ফেলা কোথায় কাজ করার দরকার নেই, ফলে অনেক শাখা mining শুরু হওয়ার আগেই কেটে ফেলা যায়।

Closure Checking

নতুন frequent itemset পেলে দুই রকম চেক করতে হয়:

- **Superset checking:** নতুনটা কি আগের কোনো closed itemset-এর superset (একই support)? — Item merging পদ্ধতিতেই স্বয়ংক্রিয়ভাবে হয়ে যায়।
- **Subset checking:** নতুনটা কি আগের কোনো closed itemset-এর subset (একই support)? — এটা চেক করতে একটা pattern-tree রাখা হয়, যেখানে এখন পর্যন্ত পাওয়া সব closed itemset জমা থাকে।

দ্রুত খোঁজার জন্য দুই-স্তরের hash index: প্রথম key = নতুন itemset-এর শেষ আইটেম, দ্বিতীয় key = তার support count।

সহজ ভাষায়: এটা অনেকটা লাইব্রেরির ক্যাটালগের মতো — প্রথমে "শেষ অক্ষর" দিয়ে ভাগ, তারপর সেই ভাগের মধ্যে "সংখ্যা" দিয়ে আরও ভাগ — এতে পুরো লাইব্রেরি না ঘেঁটেই দ্রুত খোঁজা যায়।

Closed আর Maximal itemset-এর মধ্যে অনেক মিল থাকায়, একই optimization কৌশল (item merging, sub-itemset pruning, item skipping) Maximal itemset mining-এও প্রায় একইভাবে কাজে লাগানো যায়।

৬.২.৫ ও ৬.২.৬ — সংক্ষেপে সারাংশ

বিষয়	মূল কথা
Vertical format	আইটেম ধরে TID set রাখা (উল্টো ব্যবস্থা)
Eclat	দুই itemset-এর TID set intersect করেই count পাওয়া যায় — ডাটাবেজ আবার scan লাগে না
Diffset	পুরো TID set না রেখে শুধু পার্থক্যটুকু রাখলে মেমোরি বাঁচে
Item merging	কমন প্রেফিক্স থাকলে সরাসরি merge করে closed itemset বানানো
Sub-itemset pruning	ছোট subset-এর support বড় itemset-এর সমান হলে, সেই subset বাদ
Item skipping	নিচের স্তরে একই support পেলে উপরের স্তর থেকে সেই আইটেম বাদ
Subset checking	Pattern-tree ও দুই-স্তরের hash index দিয়ে দ্রুত যাচাই

৬.৩, ৬.৩.১ ও ৬.৩.২: কোন Pattern গুলো আসলেই Interesting?

৬.৩: ভূমিকা

এতদিন আমরা শিখেছি — support আর confidence দুটো threshold পার করলেই একটা rule-কে "strong" বলা হয়। কিন্তু প্রশ্ন হলো — শুধু support-confidence পার করলেই কি rule সত্যিই "interesting" হয়?

উত্তর: না, সবসময় না! strong rule আসলে misleading (বিশ্রান্তিকর) হতে পারে, আর সেটা ঠিক করার জন্য নতুন measure লাগে।

৬.৩.১: Strong Rule মানেই Interesting নয়

বিখ্যাত উদাহরণ — কম্পিউটার গেম আর ভিডিও

AllElectronics-এ ১০,০০০টা transaction আছে। এর মধ্যে:

- ৬,০০০ টা transaction-এ কম্পিউটার গেম কেনা হয়েছে
- ৭,৫০০ টা transaction-এ ভিডিও কেনা হয়েছে
- ৪,০০০ টা transaction-এ দুটোই কেনা হয়েছে

$$\text{buys}(\text{game}) \Rightarrow \text{buys}(\text{video}) \quad [\text{support} = 40\%, \text{confidence} = 66\%]$$

$$\text{support} = 4000/10000 = 40\% \quad \text{confidence} = 4000/6000 = 66\%$$

ধরি $\text{min_sup}=30\%$, $\text{min_conf}=60\%$ — দুটোই পার হয়ে গেছে! তাই এটা একটা strong rule।

সমস্যাটা কোথায়?

ভিডিও কেনার সাধারণ সম্ভাবনা (game কেনা হোক বা না হোক) কত?

$$P(\text{video}) = 7500/10000 = 75\%$$

rule বলছে "game কিনলে video কেনার সম্ভাবনা ৬৬%"। কিন্তু আসলে এমনিতেই (game কেনা ছাড়াই) ভিডিও কেনার সম্ভাবনা ৭৫%! মানে game কেনার ফলে video কেনার সম্ভাবনা বাড়েনি, বরং কমে গেছে (৭৫% থেকে ৬৬%-এ)।

game আর video আসলে negatively associated — কিন্তু শুধু support-confidence দেখে rule-টাকে "strong" বলা হচ্ছে, যেটা সম্পূর্ণ বিভ্রান্তিকর!

সহজ analogy: একজন ডাক্তার বললেন "যারা ওষুধ X খায়, তাদের ৬৬% সুস্থ হয়ে যায়"। শুনে মনে হবে ওষুধ কাজ করছে। কিন্তু যদি জানা যায় — ওষুধ না খেলেও ৭৫% মানুষ এমনিতেই সুস্থ হয়ে যায় — তাহলে বোঝা যাবে ওষুধটা আসলে ক্ষতিই করছে!

মূল শিক্ষা: শুধু confidence দেখে rule-এর শক্তি বোঝা যায় না — কারণ এটা B-এর নিজস্ব (baseline) সম্ভাবনার সাথে তুলনা করে না। এই সমস্যা সমাধানের জন্যই দরকার correlation measure।

৬.৩.২: Association থেকে Correlation Analysis-এ যাওয়া

এখন rule-এ শুধু support আর confidence না, একটা correlation measure-ও যোগ করা হবে:

$$A \Rightarrow B \text{ [support, confidence, correlation]}$$

Correlation Measure ১: Lift

$$\text{lift}(A,B) = P(A \cup B) / (P(A) \times P(B))$$

- $P(A) \times P(B)$ — A আর B সম্পূর্ণ independent হলে, তারা একসাথে ঘটার সম্ভাব্য সম্ভাবনা
- $P(A \cup B)$ — বাস্তবে A আর B আসলে একসাথে কত ঘটছে

lift বলছে — বাস্তবে যা ঘটছে, সেটা যদি স্বাধীন হতো তার তুলনায় কতগুণ বেশি বা কম।

Lift-এর মান	মানে
< 1	Negative correlation — একটা ঘটলে অন্যটা কম ঘটে
= 1	কোনো correlation নেই — সম্পূর্ণ independent
> 1	Positive correlation — একটা ঘটলে অন্যটার সম্ভাবনা বাড়ে

আমাদের game-video উদাহরণে:

$$P(\text{game})=0.60, P(\text{video})=0.75, P(\text{game} \cup \text{video})=0.40$$

$$\text{lift} = 0.40 / (0.60 \times 0.75) = 0.40 / 0.45 = 0.89$$

যেহেতু $0.89 < 1$, তাই game আর video আসলে negatively correlated!

শর্টকাট: lift আসলে $\text{confidence}(A \Rightarrow B) / \text{support}(B)$ -এর সমান — rule-এর confidence-কে B-এর নিজস্ব বেস-রেটের সাথে ভাগ করে দিচ্ছে। ভাগফল ১-এর কম হলে, A থাকলে B-এর সম্ভাবনা কমে যাচ্ছে (বেস-রেটের চেয়ে)।

Correlation Measure ২: χ^2 (Chi-square)

Contingency table বানাই:

	game	¬game	মোট
video	4000	3500	7500
¬video	2000	500	2500
মোট	6000	4000	10000

এখানে "observed" (আসল) মান দেওয়া। এখন expected (independent হলে যা হওয়া উচিত) মান বের করি:

$$\text{expected}(\text{game}, \text{video}) = (\text{row total} \times \text{col total}) / \text{grand total} = (7500 \times 6000) / 10000 = 4500$$

একইভাবে বাকি তিনটা ঘরের expected মান বের করা যায়:

	game	¬game
video	4000 (expected: 4500)	3500 (expected: 3000)
¬video	2000 (expected: 1500)	500 (expected: 1000)

$$\chi^2 = \sum (observed - expected)^2 / expected$$

$$\begin{aligned}\chi^2 &= (4000-4500)^2/4500 + (3500-3000)^2/3000 + (2000-1500)^2/1500 + \\ &\quad (500-1000)^2/1000 \\ &= 55.6 + 83.3 + 166.7 + 250 = 555.6\end{aligned}$$

$\chi^2 = 555.6$ — এটা ১-এর চেয়ে অনেক বড়, তার মানে A আর B correlated (independent না)।

কিন্তু positive না negative? দেখতে হবে observed vs expected তুলনা করে — game আর video-এর ঘরে observed (4000) < expected (4500)। মানে বাস্তবে যতটা একসাথে হওয়ার কথা ছিল, তার চেয়ে কম হচ্ছে — এটাও negative correlation-ই বোঝাচ্ছে, lift-এর সাথে মিলেও যাচ্ছে।

সহজ Analogy দিয়ে পুরোটা মনে রাখি

ধরুন একটা স্কুলে দেখা গেলো, যারা ফুটবল খেলে, তাদের ৭০% ভালো নম্বর পায়। শুনে মনে হবে ফুটবল খেলা ভালো নম্বরের সাথে যুক্ত।

কিন্তু যদি জানা যায় — পুরো স্কুলেই এমনিতে ৮৫% ছাত্র ভালো নম্বর পায় (ফুটবল খেলুক বা না খেলুক) — তাহলে বোঝা যাবে, আসলে ফুটবল খেলা ভালো নম্বরের সম্ভাবনা কমিয়েই দিচ্ছে (৮৫% থেকে ৭০%-এ)!

Support-confidence শুধু বলবে "৭০% ভালো নম্বর পায়" — confusing। Lift বা χ^2 বলবে "এমনিতে যা হওয়ার কথা, তার চেয়ে কম হচ্ছে — তাই এটা আসলে negative সম্পর্ক"।

সংক্ষেপে সারাংশ

বিষয়	মূল কথা
৬.৩ (ভূমিকা)	শুধু support-confidence দিয়ে সব "interesting" rule ধরা যায় না
৬.৩.১	Strong rule misleading হতে পারে — confidence(66%) আসলে baseline(75%)-এর চেয়ে কম, যা negative correlation বোঝায়
৬.৩.২ — Lift	lift = $P(A \cup B) / (P(A)P(B))$; 1-এর কম = negative, 1 = independent, বেশি = positive
৬.৩.২ — χ^2	Observed vs Expected এর পার্থক্য দিয়ে correlation মাপা

মূল শিক্ষা: একটা rule "strong" হওয়া মানেই সেটা "সত্যিকারের অর্থবহ সম্পর্ক" বোঝায় না। rule-এর ভেতরের আইটেমগুলো একে অপরকে সত্যিই সাহায্য করছে না ক্ষতি করছে — সেটা বুঝতে lift বা χ^2 -এর মতো correlation measure দরকার, যেগুলো "যদি স্বাধীন হতো" তার সাথে তুলনা করে।

অধ্যায় ৬.৩.৩

Pattern Evaluation Measure-দের তুলনা — সহজ বাংলায়, সঠিক টেবিলসহ

ভূমিকা

আগের সেকশনে (৬.৩.১, ৬.৩.২) আমরা শিখেছি — শুধু support-confidence দিয়ে সব "interesting" rule ধরা যায় না, তাই lift আর χ^2 শিখেছি। এই সেকশনে আরও চারটা নতুন measure শিখবো — all confidence, max confidence, Kulczynski, cosine — আর দেখবো এই ৬টার মধ্যে কোনটা সবচেয়ে ভরসাযোগ্য।

চারটা নতুন Measure-এর সূত্র

$$all_conf(A,B) = sup(A \cup B) / \max\{sup(A), sup(B)\} = \min\{P(A/B), P(B/A)\}$$

$$max_conf(A,B) = \max\{P(A/B), P(B/A)\}$$

$$Kulc(A,B) = \frac{1}{2} (P(A/B) + P(B/A))$$

$$cosine(A,B) = sup(A \cup B) / \sqrt{sup(A) \times sup(B)} = \sqrt{P(A/B) \times P(B/A)}$$

এই চারটা measure-ই শুধু A, B, আর AUB-এর support দিয়ে নির্ধারিত হয় — মোট transaction সংখ্যা এখানে ঢেকে না। প্রতিটার মান 0 থেকে 1-এর মধ্যে।

বইয়ের আসল টেবিল (Table 6.9) — সঠিক কলাম-ক্রম

চারটা সংখ্যা এভাবে সাজানো থাকে: mc, m̄c, mc̄, m̄c̄

- **mc** = milk ও coffee দুটোই কিনেছে
- **m̄c** = milk নেই, কিন্তু coffee কিনেছে
- **mc̄** = milk কিনেছে, কিন্তু coffee নেই
- **m̄c̄** = দুটোর কোনোটাই কেনেনি (null-transaction)

Data Set	mc	m̄c	mc̄	m̄c̄	χ^2	lift	all_conf	max_conf	Kulc	cosine
D1	10,000	1,000	1,000	100,000	90557	9.26	0.91	0.91	0.91	0.91
D2	10,000	1,000	1,000	100	0	1	0.91	0.91	0.91	0.91
D3	100	1,000	1,000	100,000	670	8.44	0.09	0.09	0.09	0.09
D4	1,000	1,000	1,000	100,000	24740	25.75	0.5	0.5	0.5	0.5
D5	1,000	100	10,000	100,000	8173	9.18	0.09	0.91	0.5	0.29
D6	1,000	10	100,000	100,000	965	1.97	0.01	0.99	0.5	0.10

D1 বনাম D2 — Null-transaction বদলে দিলে কী হয়

D1

$$mc=10,000, \bar{m}c=1,000, m\bar{c}=1,000, \bar{m}\bar{c}=100,000$$

$$m \text{ (milk-এর মোট)} = mc+m\bar{c} = 10,000+1,000 = 11,000$$

$$c \text{ (coffee-এর মোট)} = mc+m\bar{c} = 10,000+1,000 = 11,000$$

$$N = 10,000+1,000+1,000+100,000 = 112,000$$

$$lift = P(mc)/(P(m)P(c)) = (10000/112000) / (11000/112000)^2 \approx 9.26 \checkmark$$

$$all_conf = mc / \max(m,c) = 10,000/11,000 \approx 0.91 \checkmark$$

D2

শুধু $m\bar{c}$ বদলে 100,000 থেকে 100 করা হলো। বাকি সব (mc , $m\bar{c}$, $m\bar{c}$) অপরিবর্তিত — মানে $m=11,000$, $c=11,000$ -ই থাকছে।

$$N = 10,000+1,000+1,000+100 = 12,100$$

$$lift = (10000/12100) / (11000/12100)^2 \approx 1.0$$

$$all_conf = 10,000/11,000 \approx 0.91 \text{ (অপরিবর্তিত!)}$$

"যারা milk কিনেছে, তাদের মধ্যে coffee-ও কেনার হার" (mc/m) — দুই ডেটাসেটেই ঠিক ৯১%, কারণ এই তিনটা সংখ্যা বদলায়নি। milk-coffee-এর আসল সম্পর্ক একটুও বদলায়নি।

কিন্তু lift 9.26 থেকে ধপ করে 1.0-এ নেমে গেলো, শুধু null-transaction বদলানোর কারণে! all_confidence কিন্তু অবিচল — 0.91-ই থাকলো।

Null-Invariance — মূল কনসেপ্ট

সংজ্ঞা: একটা measure null-invariant, যদি null-transaction ($m\bar{c}$)-এর সংখ্যা বাড়া-কমায় তার মান না বদলায়।

কেন গুরুত্বপূর্ণ? সুপারমার্কেটে দৈনিক মোট ট্রানজেকশন সংখ্যা ওঠানামা করে, কিন্তু milk-coffee-এর নিজেদের মধ্যকার সম্পর্ক তো প্রতিদিন বদলায় না। তাই ভালো measure-কে "মোট ট্রানজেকশনের ওঠানামা" প্রভাবিত করা উচিত না।

Null-invariant না	সবগুলোই null-invariant
lift, χ^2	all_conf, max_conf, Kulc, cosine

D3 ও D4 — Positive/Negative/Neutral আলাদা করা

D3

$$mc=100, \bar{m}c=1,000, m\bar{c}=1,000 \rightarrow m=1,100, c=1,100$$

$$mc/m = 100/1,100 \approx 9\% \text{ (Negative association)}$$

$$all_conf = 100/1,100 \approx 0.09 \checkmark$$

D4

$$mc=1,000, \bar{m}c=1,000, m\bar{c}=1,000 \rightarrow m=2,000, c=2,000$$

$$mc/m = 1,000/2,000 = 50\% \text{ (Neutral)}$$

$$all_conf = 1,000/2,000 = 0.50 \checkmark$$

Dataset	সম্পর্ক	all_conf (ও বাকি ৩টা)
D2	Positive (91%)	0.91 ✓
D3	Negative (9%)	0.09 ✓
D4	Neutral (50%)	0.50 ✓

চারটা null-invariant measure ঠিকভাবেই positive > neutral > negative ক্রম দেখাচ্ছে। কিন্তু lift/ χ^2 এই ক্রম রক্ষা করতে ব্যর্থ (D3-এর lift 8.44, D4-এর lift 25.75 — negative কেসের lift, neutral কেসের lift-এর চেয়ে কম হওয়া উচিত হলেও এলোমেলো আচরণ করছে কারণ এরা N-এর উপর নির্ভরশীল)।

D5 — Imbalanced ডেটা (এখানেই সবচেয়ে গুরুত্বপূর্ণ শিক্ষা)

$$mc=1,000, \bar{m}c=100, m\bar{c}=10,000, \bar{m}\bar{c}=100,000$$

$$m \text{ (milk-এর মোট)} = mc+m\bar{c} = 1,000+10,000 = 11,000$$

$$c \text{ (coffee-এর মোট)} = mc+\bar{m}c = 1,000+100 = 1,100$$

Milk অনেক বেশি জনপ্রিয় (11,000 জন), coffee কম জনপ্রিয় (মাত্র 1,100 জন)।

দিক ১ — milk কিনলে coffee-ও কিনবে কি?

$$P(c/m) = mc/m = 1,000/11,000 \approx 9\%$$

দিক ২ — coffee কিনলে milk-ও কিনবে কি?

$$P(m/c) = mc/c = 1,000/1,100 \approx 91\%$$

একই ডেটা, দুই সম্পূর্ণ বিপরীত উত্তর! কারণ — coffee কম জনপ্রিয় হওয়ায়, যে ১,০০০ জন coffee কিনেছে তাদের প্রায় সবাই-ই (৯১%) milk-ও কিনেছে। কিন্তু milk অনেক বেশি জনপ্রিয় হওয়ায়, সেই বিশাল milk-দলের তুলনায় সেই ১,০০০ জন নগণ্য অংশ (মাত্র ৯%)।

চারটা measure কী বলছে:

$$all_conf = mc / \max(m, c) = 1,000 / 11,000 \approx 0.09$$

$$max_conf = mc / \min(m, c) = 1,000 / 1,100 \approx 0.91$$

$$Kulc = (0.09 + 0.91) / 2 = 0.50$$

$$cosine = mc / \sqrt{(m \times c)} = 1,000 / \sqrt{(11,000 \times 1,100)} \approx 0.29$$

- **max_conf (0.91)** শুধু "বড়টা" দেখায় → ভুলভাবে strongly positive বলে ফেলে
- **all_conf (0.09)** শুধু "ছোটটা" দেখায় → ভুলভাবে strongly negative বলে ফেলে
- **Kulczynski (0.50)** দুটোর গড় নিয়ে neutral বলে — এই imbalanced পরিস্থিতিতে সবচেয়ে যুক্তিসঙ্গত

D6 — আরও বেশি Imbalanced

$$mc=1,000, \bar{m}c=10, m\bar{c}=100,000, \bar{m}\bar{c}=100,000$$

$$m = mc + m\bar{c} = 1,000 + 100,000 = 101,000 \quad c = mc + \bar{m}c = 1,000 + 10 = 1,010$$

এবার milk, coffee-এর চেয়ে প্রায় ১০০ গুণ বেশি জনপ্রিয় — D5-এর চেয়েও বেশি অসম।

$$P(c|m) = 1,000 / 101,000 \approx 1\% \quad P(m|c) = 1,000 / 1,010 \approx 99\%$$

$$all_conf \approx 0.01, max_conf \approx 0.99, Kulc = (0.01 + 0.99) / 2 = 0.50, cosine \approx 0.10$$

লক্ষ্য করুন — D5 আর D6 দুটোতেই Kulc = 0.50! কিন্তু বাস্তবে D6 অনেক বেশি skewed (একদিকে হেলানো)। Kulc একা এই পার্থক্য ধরতে পারছে না — তাই আরেকটা measure দরকার।

Imbalance Ratio (IR) — Kulc-এর সাথী

$$IR(A, B) = |sup(A) - sup(B)| / (sup(A) + sup(B) - sup(A \cup B))$$

D4-এ: $m=c=2,000, mc=1,000$

$$IR = |2000 - 2000| / (2000 + 2000 - 1000) = 0 \quad (\text{সম্পূর্ণ balanced})$$

D5-এ: $m=11,000, c=1,100, mc=1,000$

$$IR = |11,000 - 1,100| / (11,000 + 1,100 - 1,000) = 9,900 / 11,100 \approx 0.89$$

D6-এ: $m=101,000$, $c=1,010$, $mc=1,000$

$$IR = |101,000 - 1,010| / (101,000 + 1,010 - 1,000) = 99,990 / 101,010 \approx 0.99$$

D6-এর IR (0.99) > D5-এর IR (0.89) — সঠিকভাবেই বলছে D6 বেশি skewed, যেটা Kulc একা বলতে পারেনি।

সবচেয়ে সহজ Analogy

ধরুন একটা ক্লাসে অঙ্ক (যেটাতে প্রায় সবাই ভর্তি হয়, খুব জনপ্রিয়) আর সংগীত (যেটাতে অল্প কিছু ছাত্র পড়ে, কম জনপ্রিয়) — এই দুই বিষয়ের সম্পর্ক দেখছেন।

যারা সংগীত পড়ে (ছোট দল), তাদের প্রায় সবাই-ই অঙ্কও পড়ে (বড় দল) → এই দিক থেকে % বেশি। কিন্তু যারা অঙ্ক পড়ে (বড় দল), তাদের অল্প কয়েকজনই সংগীতও পড়ে → এই দিক থেকে % কম।

max_conf ছোট-দল-এর দিক থেকে (বেশি %) দেখাবে, all_conf বড়-দল-এর দিক থেকে (কম %) দেখাবে। Kulczynski দুটোর গড় করে একটা ভারসাম্যপূর্ণ উত্তর দেয়, আর IR বলে দেয় অঙ্ক-সংগীতের জনপ্রিয়তার পার্থক্য কতটা বিশাল।

চূড়ান্ত সারাংশ

Measure	Null-invariant?	মূল সমস্যা/সুবিধা
Lift	X না	Null-transaction বদলালে ভুল ফলাফল দেয় (D1 vs D2)
χ^2	X না	একই সমস্যা
All confidence	✓ হ্যাঁ	Imbalanced ডেটায় শুধু "ছোট দিক" দেখায়
Max confidence	✓ হ্যাঁ	Imbalanced ডেটায় শুধু "বড় দিক" দেখায়
Cosine	✓ হ্যাঁ	মারামাতি আচরণ, ব্যাখ্যা একটু জটিল
Kulczynski	✓ হ্যাঁ	দুটোর গড়, সবচেয়ে ভারসাম্যপূর্ণ, কিন্তু skewness ধরে না

বইয়ের চূড়ান্ত সুপারিশ: বড় ডেটাসেটে সাধারণত অনেক null-transaction থাকে, তাই lift ও χ^2 এড়িয়ে চলাই ভালো। এদের বদলে Kulczynski + Imbalance Ratio (IR) একসাথে ব্যবহার করাই সবচেয়ে যুক্তিসঙ্গত — কারণ এটা null-invariant, ভারসাম্যপূর্ণ, আর সাথে ডেটার skewness সম্পর্কেও তথ্য দেয়।

এই নোটগুলো Data Mining: Concepts and Techniques (Han, 3rd Edition) বইয়ের অধ্যায় ৬ অবলম্বনে তৈরি।